

program.md Experimental Variants

Autoresearch — Random Forest, Gradient Boosting & Hyperparameter Tuning

Research Course Project | May 25, 2026

Purpose. This document collects all `program.md` variants used in the three studies of the autoresearch research project. Each variant is a complete, drop-in replacement for Karpathy's original `program.md` file and follows the same structural conventions (Setup, Experimentation, Output format, Logging, Experiment loop) so the agent can execute it without modification to the surrounding codebase. The variants differ only in the dimension under investigation for each study.

How to use. Copy the listing for the desired variant into the repository root as `program.md`, then launch the agent as usual. Do *not* modify `prepare.py` or `train.py` before the run; the agent reads the current state of `train.py` as the baseline on its first experiment.

Naming convention. Variant IDs follow the pattern `<STUDY>-<ALGORITHM>-<LEVEL>`, e.g. `S1-RF-A` is Study 1, Random Forest, Level A (narrow/strict/terse). Variants `S3-RF-HP` and `S3-GBM-HP` are the selected best variants used in the head-to-head hyperparameter tuning comparison of Study 3.

Contents

1	Factorial Design Matrix	2
2	Study 1 — Random Forest Variants	5
2.1	S1-RF-1 Narrow · Strict · No hint · Single metric · Terse	5
2.2	S1-RF-2 Broad · Strict · No hint · Multi-metric · Terse	7
2.3	S1-RF-3 Narrow · Permissive · No hint · Multi-metric · Verbose .	9
2.4	S1-RF-4 Broad · Permissive · No hint · Single metric · Verbose .	13
2.5	S1-RF-5 Narrow · Strict · Sequential hint · Multi-metric · Verbose	16
2.6	S1-RF-6 Broad · Strict · Sequential hint · Single metric · Verbose	19
2.7	S1-RF-7 Narrow · Permissive · Sequential hint · Single metric · Terse	22
2.8	S1-RF-8 Broad · Permissive · Sequential hint · Multi-metric · Terse	24
3	Study 2 — Gradient Boosting Variants	26
3.1	S2-GBM-1 Narrow · Strict · No hint · Single metric · Terse . . .	26
3.2	S2-GBM-2 Broad · Strict · No hint · Multi-metric · Terse	28
3.3	S2-GBM-5 Narrow · Strict · Sequential hint · Multi-metric · Verbose	30
3.4	S2-GBM-8 Broad · Permissive · Sequential hint · Multi-metric · Terse	33
4	Study 3 — Hyperparameter Tuning Comparison Variants	35
4.1	S3-RF-HP Best RF variant for HP tuning comparison	35
4.2	S3-GBM-HP Best GBM variant for HP tuning comparison	38
5	Quick Reference Summary	41

1. Factorial Design Matrix

The table below summarises the Resolution IV fractional factorial design (2^{5-2} , 8 runs per algorithm) used across Studies 1 and 2. A + denotes the high level of the dimension; a – denotes the low level. Generators: $D = AB$, $E = AC$.

Variant	A Scope	B Constraints	C Strategy hint	D Eval criterion	E Verbosity	Label
S*-RF/GBM-1	—	—	—	—	—	Narrow, strict, none, single, terse
S*-RF/GBM-2	+	—	—	+	—	Broad, strict, none, multi, terse
S*-RF/GBM-3	—	+	—	+	+	Narrow, permis- sive, none, multi, verbose
S*-RF/GBM-4	+	+	—	—	+	Broad, permis- sive, none, single, verbose
S*-RF/GBM-5	—	—	+	+	+	Narrow, strict, se- quential, multi, verbose
S*-RF/GBM-6	+	—	+	—	+	Broad, strict, se- quential, single, verbose
S*-RF/GBM-7	—	+	+	—	—	Narrow, permis- sive, sequen- tial, single, terse
S*-RF/GBM-8	+	+	+	+	—	Broad, permis- sive, sequen- tial, multi, terse

Dimension levels: **Scope** (− = narrow named HPs, + = open-ended); **Constraints** (− = strict prohibitions, + = permissive guidance only); **Strategy hint** (− = agent chooses, + = sequential one-at-a-time); **Eval criterion** (− = single metric AUC, + = multi-metric AUC + complexity); **Verbosity** (− = ≈ 120 words, + = ≈ 450 words).

2. Study 1 — Random Forest Variants

2.1 S1-RF-1 Narrow · Strict · No hint · Single metric · Terse

Variant ID: S1-RF-1 **Algorithm:** Random Forest **Scope:** Narrow **Strategy:** None (agent chooses) **Dimension tested:** Scope × Constraints × Verbosity baseline

Listing 1: S1-RF-1: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Setup

Work with the user to:

1. Agree on a run tag (e.g. 'rf1'). Create branch
   'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py'.
3. Verify data exists in '~/.cache/autoresearch/'. If not,
   ask the user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with the header row only.
5. Confirm setup and begin.

## Experimentation

Each run uses 'uv run train.py'. The time budget is 5
minutes wall-clock training
time. The only file you may edit is 'train.py'.

You may only modify these hyperparameters of the
RandomForestClassifier:
- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [3, 20] or None
- 'min_samples_leaf': integer in [1, 20]
- 'max_features': one of {"sqrt", "log2", 0.3, 0.5, 0.7}

Do NOT change the model class, the dataset, the train/val
split, or any code in
'prepare.py'. Do NOT install new packages. Do NOT modify the
evaluation function.

The goal is to minimise val_loss (lower is better). Run one
change at a time.
```

```
## Output format

After each run the script prints:

'''
val_loss:          0.2314
training_seconds: 300.1
peak_vram_mb:      1820.4
'''

Extract with: 'grep "^val_loss:" run.log'

## Logging

Log every experiment to 'results.tsv' (tab-separated).
Schema:

'''
commit  val_loss  memory_gb  status  description
'''

Status is 'keep', 'discard', or 'crash'. Use 0.000000 / 0.0
for crashes.
Do not commit 'results.tsv'.

## Experiment loop

LOOP FOREVER:

1. Inspect current branch and 'train.py'.
2. Edit one hyperparameter in 'train.py'.
3. 'git commit -m "<short description>"'
4. 'uv run train.py > run.log 2>&1'
5. 'grep "^val_loss:" run.log'
6. If grep is empty, run 'tail -n 50 run.log', attempt a
   fix, re-run once.
   If still broken, log crash and move on.
7. Record in 'results.tsv'.
8. If val_loss improved (lower): keep the commit and advance.
   If not: 'git reset --hard HEAD~1' and discard.

NEVER STOP to ask the user for permission to continue. Run
until manually interrupted.
```

2.2 S1-RF-2 Broad · Strict · No hint · Multi-metric · Terse

Variant ID: S1-RF-2 **Algorithm:** Random Forest **Scope:** Broad **Strategy:** None (agent chooses) **Dimension tested:** Scope: open-ended; Eval: multi-metric (AUC + complexity)

Listing 2: S1-RF-2: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Setup

1. Agree on a run tag. Create branch 'autoresearch/<tag>'
   from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py'.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
   user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with header only.
5. Confirm and begin.

## Experimentation

Each run uses 'uv run train.py'. Fixed 5-minute wall-clock
training budget.
The only file you may edit is 'train.py'.

You may modify anything in 'train.py' that could improve
model quality: all
hyperparameters, preprocessing steps, feature selection,
ensemble composition,
class weighting, or calibration. There is no predefined list
of allowed changes.

Do NOT modify 'prepare.py', install new packages, or change
the evaluation function.

The primary goal is to minimise val_loss. A secondary goal
is to keep the model
simple: prefer solutions with fewer trees or shallower depth
when val_loss is equal.
When deciding whether to keep a change, weigh both
objectives: a 0.001 improvement
in val_loss at the cost of doubling model size may not be
worth it. A simplification
that preserves val_loss is always worth keeping.
```

```
## Output format

'''
val_loss:          0.2314
training_seconds: 300.1
peak_vram_mb:      1820.4
num_params_M:      0.4
'''

Extract: 'grep "^val_loss:|^num_params_M:" run.log'

## Logging

Log to 'results.tsv' (tab-separated):

'''
commit  val_loss  memory_gb  status  description
'''

Do not commit 'results.tsv'.

## Experiment loop

LOOP FOREVER:

1. Inspect current 'train.py'.
2. Formulate a hypothesis and edit 'train.py'.
3. 'git commit -m "<hypothesis and expected effect>"'
4. 'uv run train.py > run.log 2>&1'
5. Extract val_loss and model size. Evaluate both objectives.
6. On crash: read log tail, fix if trivial, else log crash
   and move on.
7. Record in 'results.tsv'.
8. Keep if val_loss improved or simplicity improved without
   loss degradation.
   Otherwise 'git reset --hard HEAD~1'.

NEVER STOP. Run until manually interrupted.
```


2.3 S1-RF-3 Narrow · Permissive · No hint · Multi-metric · Verbose

Variant ID: S1-RF-3 **Algorithm:** Random Forest **Scope:** Narrow **Strategy:** None (agent chooses) **Dimension tested:** Constraints: permissive; Eval: multi-metric; Verbosity: high

Listing 3: S1-RF-3: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Overview

This is an autonomous ML experimentation session. Your role
  is that of a
thorough and curious ML researcher. You will iteratively
  improve a Random Forest
classifier by proposing and testing hypotheses about the
  model configuration.
Document your reasoning carefully in every commit message so
  that the experiment
log can be analysed after the run.

## Setup

Work with the user to complete the following steps before
  beginning:

1. Agree on a run tag (e.g. 'rf3'). The branch
  'autoresearch/<tag>' must not
  already exist. Create it from master: 'git checkout -b
  autoresearch/<tag>'.
2. Read the following files in full: 'README.md',
  'prepare.py', and 'train.py'.
  Understand the data pipeline, the evaluation logic, and
  the current model
  configuration before proposing any changes.
3. Confirm that '~/.cache/autoresearch/' contains the
  required data shards and
  tokenizer. If the cache is empty, inform the user and ask
  them to run
  'uv run prepare.py' before proceeding.
4. Create 'results.tsv' with the header row only. The
  baseline will be recorded
  after the first run.
5. Confirm that setup is complete and await the user's
  go-ahead.

## What you may modify
```

```
You may modify any of the following hyperparameters of the
    RandomForestClassifier
in 'train.py':
- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [3, 20] or None (unlimited)
- 'min_samples_leaf': integer in [1, 20]
- 'min_samples_split': integer in [2, 20]
- 'max_features': one of {"sqrt", "log2", 0.3, 0.5, 0.7, 1.0}
- 'bootstrap': True or False
- 'class_weight': None, "balanced", or "balanced_subsample"
```

You are encouraged to make changes outside this list if you have a well-reasoned hypothesis, but document the rationale clearly. The spirit of the constraint is to keep experiments interpretable, not to prevent creativity.

What you must not do

Do not modify 'prepare.py' under any circumstances. It contains the evaluation logic and data pipeline, and altering it would invalidate the comparison with other runs. Do not install new packages. Do not change the train/validation split or the random seed used for data loading.

Evaluation criteria

The primary metric is `val_loss` (lower is better). As a secondary consideration, prefer configurations that are less computationally expensive: fewer trees, shallower depth, and lower inference time are all desirable when `val_loss` is not degraded. Think of the secondary criterion as a tiebreaker rather than a competing objective: if two configurations produce the same `val_loss`, prefer the simpler one.

Output format

The training script prints a summary at completion:

```
'''
val_loss:          0.2314
```

```

training_seconds: 300.1
peak_vram_mb:      1820.4
num_params_M:      0.4
'''

Extract the key metrics: 'grep "^val_loss:|^num_params_M:"
    run.log'

## Logging

Record every experiment in 'results.tsv' (tab-separated, not
    comma-separated).
Schema:

'''
commit    val_loss    memory_gb    status    description
'''

The description field should be a concise summary of the
    hypothesis tested (e.g.
    "increase n_estimators from 100 to 300 to reduce variance").
    Use status 'keep',
    'discard', or 'crash'. Use 0.000000 / 0.0 for crashes. Do
    not add 'results.tsv'
    to git.

## Experiment loop

LOOP FOREVER until manually stopped:

1. Review the current state of 'train.py' and the recent
    'results.tsv' entries.
2. Formulate a clear hypothesis about what change is likely
    to improve val_loss
    and why. Consider the history of experiments to avoid
    repeating failed ideas.
3. Edit 'train.py' to implement the hypothesis.
4. Commit: 'git commit -m "<hypothesis: expected effect>"'
5. Run: 'uv run train.py > run.log 2>&1'
6. Extract results: 'grep "^val_loss:|^num_params_M:"
    run.log'
7. If grep returns nothing, the run crashed. Read the last
    50 lines of the log,
    fix trivial errors and re-run once. If the idea is
    fundamentally broken, log
    a crash entry and move to the next hypothesis.
8. Record in 'results.tsv'.

```

```
9. If val_loss improved: keep the commit. If not: 'git reset
--hard HEAD~1'.
```

```
NEVER STOP to ask the user for permission to continue. If
you run out of obvious
ideas, re-read the in-scope files, review near-misses in the
log, and try bolder
combinations. You are autonomous. Run until interrupted.
```

2.4 S1-RF-4 Broad · Permissive · No hint · Single metric · Verbose

Variant ID: S1-RF-4 **Algorithm:** Random Forest **Scope:** Broad **Strategy:** None (agent chooses) **Dimension tested:** Scope: open-ended; Constraints: permissive; Verbosity: high

Listing 4: S1-RF-4: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Overview

This is an open-ended autonomous research session. You are a
creative ML researcher
with full latitude to improve a Random Forest classifier by
any means you judge
promising. There is no predefined list of allowed changes.
Your only hard constraint
is the evaluation function defined in 'prepare.py', which
you must not touch.

Approach this as you would a competitive ML benchmark:
explore widely, follow
promising leads, and do not limit yourself to textbook
hyperparameter tuning.
Feature engineering, preprocessing, ensemble composition,
calibration, and any
other technique that can be expressed as a change to
'train.py' is fair game.

## Setup

1. Agree on a run tag. Create branch 'autoresearch/<tag>'
   from master.
2. Read 'README.md', 'prepare.py', and 'train.py' in full
   before proposing
   any changes.
3. Verify '~/cache/autoresearch/' contains required data.
   If not, ask the user
   to run 'uv run prepare.py'.
4. Create 'results.tsv' with the header row only.
5. Confirm readiness and begin as soon as the user gives the
   go-ahead.

## Experimentation

Each run uses 'uv run train.py'. Fixed 5-minute wall-clock
training budget.
```

You may edit 'train.py' freely. You may not modify
 'prepare.py', install new
 packages, or change the evaluation function.

The sole objective is to minimise val_loss. Do not apply any
 secondary criteria
 unless they help val_loss directly. If a complex change
 improves val_loss, keep
 it even if it increases model size. If a simplification does
 not degrade val_loss,
 keep it too but simplicity is not a goal in itself here.

When you feel you have exhausted incremental improvements,
 try qualitatively
 different approaches: change the feature representation, the
 ensemble strategy,
 or the training objective. Consult any ML knowledge you have
 about Random Forests
 and ensemble methods, and apply it directly.

Output format

```
'''
val_loss:          0.2314
training_seconds:  300.1
peak_vram_mb:      1820.4
'''
```

Extract: 'grep "^val_loss:" run.log'

Logging

Log to 'results.tsv' (tab-separated):

```
'''
commit  val_loss  memory_gb  status  description
'''
```

Status: 'keep', 'discard', or 'crash'. Crash entries use
 0.000000 / 0.0.

Do not commit 'results.tsv'.

Experiment loop

LOOP FOREVER:

1. Review 'train.py' and recent 'results.tsv' history.

2. Propose and implement the change you judge most likely to improve `val_loss`.
Write a clear commit message explaining the idea and the expected effect.
3. `'git commit -m "<idea and rationale>"'`
4. `'uv run train.py > run.log 2>&1'`
5. `'grep "^val_loss:" run.log'`
6. On crash: `'tail -n 50 run.log'`, fix if trivial, else log crash and continue.
7. Record in `'results.tsv'`.
8. Keep if `val_loss` improved; `'git reset --hard HEAD~1'` otherwise.

NEVER STOP. If you run out of ideas, think harder: revisit near-misses, try combinations, attempt radical changes. Run until manually interrupted.

2.5 S1-RF-5 Narrow · Strict · Sequential hint · Multi-metric · Verbose

Variant ID: S1-RF-5 **Algorithm:** Random Forest **Scope:** Narrow **Strategy:** Sequential (one HP at a time) **Dimension tested:** Strategy hint: explicit sequential; Eval: multi-metric; Verbosity: high

Listing 5: S1-RF-5: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Overview

This is a structured, sequential hyperparameter optimisation
session. You will
improve a Random Forest classifier by changing exactly one
hyperparameter per
experiment. This discipline is intentional: it produces a
clean causal record of
which changes are responsible for which improvements, and it
prevents confounded
results that arise from changing multiple things at once.

## Setup

1. Agree on a run tag (e.g. 'rf5'). Create branch
   'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py' in full.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
   the user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with the header row only.
5. Confirm setup and await go-ahead.

## What you may modify

Change exactly one of the following hyperparameters per
experiment:
- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [3, 20] or None
- 'min_samples_leaf': integer in [1, 20]
- 'min_samples_split': integer in [2, 20]
- 'max_features': one of {"sqrt", "log2", 0.3, 0.5, 0.7}
- 'bootstrap': True or False

Each experiment must change exactly one hyperparameter from
the current best
configuration. Do not bundle changes. If you want to test an
```



```

    interaction between
two hyperparameters, test each one individually first.

## What you must not do

Do not modify 'prepare.py', install packages, or change the
    evaluation function.
Do not change the model class, the dataset, or the
    train/validation split.

## Evaluation criteria

Optimise two objectives jointly:
1. Minimise val_loss (primary).
2. Minimise model complexity, defined as 'n_estimators *
    max_depth' where
    max_depth defaults to 20 if set to None (secondary).

When deciding whether to keep a change, compute a weighted
    score:
    score = val_loss + 0.0001 * complexity_proxy

Keep the change if the score improves. This weighting means
    a val_loss reduction
of 0.001 is worth approximately a 10x reduction in the
    complexity proxy.

## Output format

'''
val_loss:          0.2314
training_seconds: 300.1
peak_vram_mb:      1820.4
num_params_M:      0.4
'''

Extract: 'grep "^val_loss:|num_params_M:" run.log'

## Logging

Log to 'results.tsv' (tab-separated):

'''
commit  val_loss  memory_gb  status  description
'''

In the description field, always state: (a) which

```

```
hyperparameter was changed,  
(b) from what value to what value, and (c) the composite  
score. Example:  
"n_estimators 100->200, val_loss 0.2314->0.2287, score  
0.2289->0.2289 keep"  
  
Do not commit 'results.tsv'.  
  
## Experiment loop  
  
LOOP FOREVER:  
  
1. Review 'train.py' and recent history. Identify which  
hyperparameter to probe  
next and in which direction, based on the pattern of past  
results.  
2. Change exactly one hyperparameter.  
3. 'git commit -m "<HP changed: old->new, hypothesis>"'  
4. 'uv run train.py > run.log 2>&1'  
5. 'grep "^val_loss:|^num_params_M:" run.log'  
6. Compute the composite score. On crash: read log, fix if  
trivial, else log crash.  
7. Record in 'results.tsv' with the full description format  
above.  
8. Keep if composite score improved; 'git reset --hard  
HEAD~1' otherwise.  
  
NEVER STOP. Run until manually interrupted.
```

2.6 S1-RF-6 Broad · Strict · Sequential hint · Single metric · Verbose

Variant ID: S1-RF-6 **Algorithm:** Random Forest **Scope:** Broad **Strategy:** Sequential (one change at a time) **Dimension tested:** Scope: open-ended with sequential discipline

Listing 6: S1-RF-6: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Overview

This session combines open-ended exploration with strict
experimental discipline.
You may propose any change to the model or training pipeline
in 'train.py', but
each experiment must implement exactly one coherent change.
A "coherent change" is
a single, describable modification: changing one
hyperparameter, adding one
preprocessing step, or replacing one component of the
pipeline. Changes that affect
multiple aspects of the model at once are not permitted.

This discipline ensures the experiment log remains
interpretable even when the
search space is open-ended.

## Setup

1. Agree on a run tag (e.g. 'rf6'). Create branch
   'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py' in full.
3. Verify data in '~/cache/autoresearch/'. If absent, ask
   user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with header only.
5. Confirm and begin.

## Experimentation scope

You may modify anything in 'train.py' that could improve
val_loss: hyperparameters,
preprocessing, feature engineering, class weighting,
calibration, or any other
technique expressible as a code change. There is no
predefined list of allowed
```

```
modifications.

Do NOT modify 'prepare.py', install new packages, or change
the evaluation function.
Do NOT change more than one aspect of the model per
experiment.

The sole objective is to minimise val_loss.

## Output format

'''
val_loss:          0.2314
training_seconds:  300.1
peak_vram_mb:      1820.4
'''

Extract: 'grep "^val_loss:" run.log'

## Logging

Log to 'results.tsv' (tab-separated):

'''
commit  val_loss  memory_gb  status  description
'''

Do not commit 'results.tsv'.

## Experiment loop

LOOP FOREVER:

1. Review 'train.py' and recent results. Identify the single
   most promising
   change to try next and why.
2. Implement exactly one change.
3. 'git commit -m "<one-line description of the single
   change made>"'
4. 'uv run train.py > run.log 2>&1'
5. 'grep "^val_loss:" run.log'
6. On crash: 'tail -n 50 run.log', fix if trivial, else log
   crash and continue.
7. Record in 'results.tsv'.
8. Keep if val_loss improved; 'git reset --hard HEAD~1'
   otherwise.
```

```
NEVER STOP. Run until manually interrupted.
```

2.7 S1-RF-7 Narrow · Permissive · Sequential hint · Single metric · Terse

Variant ID: S1-RF-7 **Algorithm:** Random Forest **Scope:** Narrow **Strategy:** Sequential (one HP at a time) **Dimension tested:** Constraints: permissive; Verbosity: terse

Listing 7: S1-RF-7: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Setup

1. Agree on a run tag. Create branch 'autoresearch/<tag>'
   from master.
2. Read 'README.md', 'prepare.py', and 'train.py'.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
   user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with header only.
5. Confirm and begin.

## Experimentation

Each run: 'uv run train.py'. Fixed 5-minute budget. Edit
only 'train.py'.

Change one hyperparameter per experiment from:
- 'n_estimators' [50, 500], 'max_depth' [3, 20 or None],
  'min_samples_leaf' [1, 20], 'max_features' {sqrt, log2,
  0.3, 0.5}.

Changes outside this list are welcome if well-motivated.
Document the reason.

Do NOT touch 'prepare.py' or the evaluation function. New
packages not permitted.

Goal: minimise val_loss only.

## Output / Logging

Extract: 'grep "^val_loss:" run.log'
Log to 'results.tsv' (tab-separated): 'commit  val_loss
memory_gb  status  description'
Do not commit 'results.tsv'.

## Loop
```

```
LOOP FOREVER: inspect -> change one HP -> commit -> run ->  
    extract ->  
log -> keep if improved / reset if not. NEVER STOP.
```

2.8 S1-RF-8 Broad · Permissive · Sequential hint · Multi-metric · Terse

Variant ID: S1-RF-8 **Algorithm:** Random Forest **Scope:** Broad **Strategy:** Sequential (one change at a time) **Dimension tested:** All high-level dimensions; terse formulation

Listing 8: S1-RF-8: file to save as program.md

```
# autoresearch      Random Forest Optimisation

## Setup

1. Agree on tag. Branch 'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py', 'train.py'.
3. Check '~/.cache/autoresearch/'. If empty, ask user to run
   'uv run prepare.py'.
4. Create 'results.tsv' header only.
5. Confirm and go.

## Experimentation

'uv run train.py'. 5-minute budget. Edit only 'train.py'.

Open scope: any change to model, preprocessing, features, or
training procedure.
One coherent change per experiment. No new packages. Do not
touch 'prepare.py'.

Two objectives: (1) minimise val_loss; (2) prefer simpler
models when val_loss
is equal. Keep a change if it improves val_loss OR achieves
the same val_loss
with lower complexity.

## Output / Logging

Extract: 'grep "^val_loss:|^num_params_M:" run.log'
Log to 'results.tsv': 'commit val_loss memory_gb status
description'
Do not commit 'results.tsv'.

## Loop

LOOP FOREVER: inspect -> one change -> commit -> run ->
extract both metrics ->
log -> keep if either objective improves without degrading
the other / reset if not.
```


NEVER STOP .

3. Study 2 — Gradient Boosting Variants

The eight Gradient Boosting variants replicate the factorial design of Study 1. The setup, output, logging, and loop sections are identical to their Study 1 counterparts; only the allowed hyperparameter set and any algorithm-specific guidance change. For brevity, variants S2-GBM-3 through S2-GBM-6 show the GBM-specific sections only, with a note that the surrounding scaffold is unchanged from the corresponding Study 1 variant.

3.1 S2-GBM-1 Narrow · Strict · No hint · Single metric · Terse

Variant ID: S2-GBM-1 **Algorithm:** Gradient Boosting (XGBoost) **Scope:** Narrow
Strategy: None **Dimension tested:** Scope $\times$ Constraints $\times$ Verbosity baseline for GBM

Listing 9: S2-GBM-1: file to save as program.md

```
# autoresearch      Gradient Boosting Optimisation

## Setup

1. Agree on a run tag (e.g. 'gbm1'). Create branch
   'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py'.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
   user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with header only.
5. Confirm and begin.

## Experimentation

Each run: 'uv run train.py'. Fixed 5-minute wall-clock
budget. Edit only 'train.py'.

You may only modify these hyperparameters of the
XGBClassifier:
- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [2, 10]
- 'learning_rate': float in [0.01, 0.3]
- 'subsample': float in [0.5, 1.0]
- 'colsample_bytree': float in [0.5, 1.0]

Do NOT change the model class, the dataset, the train/val
split, 'prepare.py',
or the evaluation function. Do NOT install new packages. One
change per experiment.
```

```
Goal: minimise val_loss (lower is better).

## Output format

'''
val_loss:          0.2201
training_seconds: 300.1
peak_vram_mb:      920.3
'''

Extract: 'grep "^val_loss:" run.log'

## Logging

Log to 'results.tsv' (tab-separated):

'''
commit  val_loss  memory_gb  status  description
'''

Status: 'keep', 'discard', or 'crash'. Crash: 0.000000 /
        0.0. Do not commit file.

## Experiment loop

LOOP FOREVER:

1. Inspect 'train.py' and recent results.
2. Change exactly one hyperparameter.
3. 'git commit -m "<HP: old->new>"'
4. 'uv run train.py > run.log 2>&1'
5. 'grep "^val_loss:" run.log'
6. On crash: 'tail -n 50 run.log', fix once if trivial, else
   log crash.
7. Record in 'results.tsv'.
8. Keep if improved; 'git reset --hard HEAD~1' otherwise.

NEVER STOP. Run until manually interrupted.
```

3.2 S2-GBM-2 Broad · Strict · No hint · Multi-metric · Terse

Variant ID: S2-GBM-2 **Algorithm:** Gradient Boosting (XGBoost) **Scope:** Broad
Strategy: None **Dimension tested:** Scope: open-ended; Eval: multi-metric

Listing 10: S2-GBM-2: file to save as `program.md`

```
# autoresearch      Gradient Boosting Optimisation

## Setup

1. Agree on tag. Branch 'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py', 'train.py'.
3. Verify data in '~/.cache/autoresearch/'. Run 'uv run
   prepare.py' if absent.
4. Create 'results.tsv' header only.
5. Confirm and go.

## Experimentation

'uv run train.py'. 5-minute budget. Edit only 'train.py'.

Open scope: modify anything in 'train.py'. No predefined HP
list. This includes
all XGBoost hyperparameters, early stopping strategies,
preprocessing, and feature
selection. Do not touch 'prepare.py', install packages, or
change the eval function.

Two objectives: (1) minimise val_loss; (2) keep the model
simple      prefer fewer
trees and lower depth when val_loss is equal. A
simplification that preserves
val_loss is always a keep. A complexity increase requires a
meaningful val_loss gain.

IMPORTANT for GBM: learning_rate and n_estimators interact
strongly. If you lower
learning_rate, consider increasing n_estimators in the same
experiment only if
the individual changes have been tested first; otherwise
test them separately.

## Output / Logging

Extract: 'grep "^val_loss:|~num_params_M:" run.log'
Log to 'results.tsv': 'commit val_loss memory_gb status
description'
```

```
Do not commit 'results.tsv'.

## Loop

LOOP FOREVER: inspect -> change -> commit -> run -> extract
    both metrics -> log ->
keep if either objective improves without degrading the
    other / reset if not.
NEVER STOP.
```

3.3 S2-GBM-5 Narrow · Strict · Sequential hint · Multi-metric · Verbose

Variant ID: S2-GBM-5 **Algorithm:** Gradient Boosting (XGBoost) **Scope:** Narrow
Strategy: Sequential **Dimension tested:** Strategy hint: explicit sequential with interaction guidance

Listing 11: S2-GBM-5: file to save as program.md

```
# autoresearch      Gradient Boosting Optimisation

## Overview

This is a structured sequential hyperparameter optimisation
session for a Gradient
Boosting classifier. You will change exactly one
hyperparameter per experiment.
This discipline is especially important for Gradient
Boosting, where parameter
interactions are strong: isolating individual effects is the
only reliable way to
build a causal understanding of which changes are
responsible for which improvements.

Pay particular attention to the learning_rate / n_estimators
trade-off. These two
parameters are closely coupled: a lower learning_rate
generally requires more
estimators to achieve comparable performance. Test each
individually before drawing
conclusions about their joint effect.

## Setup

1. Agree on a run tag (e.g. 'gbm5'). Create branch
   'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
   'train.py' in full.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
   user to run
   'uv run prepare.py'.
4. Create 'results.tsv' with header only.
5. Confirm readiness and await go-ahead.

## What you may modify

Change exactly one of the following per experiment:
- 'n_estimators': integer in [50, 500]
```

```

- 'max_depth': integer in [2, 10]
- 'learning_rate': float in [0.005, 0.3]
- 'subsample': float in [0.5, 1.0]
- 'colsample_bytree': float in [0.5, 1.0]
- 'reg_alpha': float in [0.0, 1.0] (L1 regularisation)
- 'reg_lambda': float in [0.5, 5.0] (L2 regularisation)

Do NOT change more than one hyperparameter per experiment.
  Do NOT change the model
class, dataset, train/val split, 'prepare.py', or evaluation
  function. No new packages.

## Evaluation criteria

Optimise two objectives:
1. Minimise val_loss (primary).
2. Minimise a complexity proxy: 'n_estimators * max_depth'
   (secondary).

Composite score: 'score = val_loss + 0.0001 * (n_estimators
  * max_depth)'

Keep if composite score improves. Record both val_loss and
  composite score in the
description column.

## Output format

'''
val_loss:          0.2201
training_seconds: 300.1
peak_vram_mb:      920.3
num_params_M:      0.2
'''

Extract: 'grep "^val_loss:|^num_params_M:" run.log'

## Logging

Log to 'results.tsv' (tab-separated):

'''
commit  val_loss  memory_gb  status  description
'''

Description format: "learning_rate 0.1->0.05, val_loss
  0.2201->0.2183, score

```

```
0.2201->0.2184 keep"

Do not commit 'results.tsv'.

## Experiment loop

LOOP FOREVER:

1. Review 'train.py' and recent history. Decide which single
   hyperparameter to
   probe next based on the pattern of past results and the
   interaction structure
   described above.
2. Change exactly one hyperparameter.
3. 'git commit -m "<HP: old->new, hypothesis>"'
4. 'uv run train.py > run.log 2>&1'
5. 'grep "^val_loss:|^num_params_M:" run.log'
6. Compute composite score. On crash: read log tail, fix if
   trivial, else log crash.
7. Record in 'results.tsv' with full description format.
8. Keep if composite score improved; 'git reset --hard
   HEAD~1' otherwise.

NEVER STOP. Run until manually interrupted.
```


3.4 S2-GBM-8 Broad · Permissive · Sequential hint · Multi-metric · Terse

Variant ID: S2-GBM-8 **Algorithm:** Gradient Boosting (XGBoost) **Scope:** Broad
Strategy: Sequential (one change at a time) **Dimension tested:** All high-level dimensions active; GBM interaction-aware

Listing 12: S2-GBM-8: file to save as program.md

```
# autoresearch      Gradient Boosting Optimisation

## Setup

1. Agree on tag. Branch 'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py', 'train.py'.
3. Check '~/.cache/autoresearch/'. Empty? Ask user to run
   'uv run prepare.py'.
4. Create 'results.tsv' header only.
5. Confirm and go.

## Experimentation

'uv run train.py'. 5-minute budget. Edit only 'train.py'.

Open scope: any change to model, preprocessing, features, or
training logic.
One coherent change per experiment. No new packages. Do not
touch 'prepare.py'.

GBM note: the learning_rate / n_estimators trade-off is the
most important
interaction in this model. Test them individually before
combining.

Two objectives: (1) minimise val_loss; (2) prefer simpler
models at equal
val_loss. Keep if val_loss improves OR simplicity improves
without val_loss loss.

## Output / Logging

Extract: 'grep "^val_loss:|num_params_M:" run.log'
Log to 'results.tsv': 'commit val_loss memory_gb status
description'
Do not commit 'results.tsv'.

## Loop
```

```
LOOP FOREVER: inspect -> one coherent change -> commit ->  
    run ->  
extract both metrics -> log -> keep / reset. NEVER STOP.
```

4. Study 3 — Hyperparameter Tuning Comparison Variants

Study 3 uses the single best-performing `program.md` from each algorithm’s Study 1/2 runs. The two variants below are designed to be that best-performer candidate based on prior literature expectations, and will be confirmed or replaced by the actual Study 1/2 results before Study 3 begins. They are also the most explicit about *budget tracking*, since the head-to-head comparison requires a precisely matched experiment count.

4.1 S3-RF-HP Best RF variant for HP tuning comparison

Variant ID: S3-RF-HP **Algorithm:** Random Forest **Scope:** Narrow **Strategy:** Sequential (one HP at a time) **Dimension tested:** Budget-tracked; used in Study 3 comparison

Listing 13: S3-RF-HP: file to save as `program.md`

```
# autoresearch      Random Forest Hyperparameter Tuning
  (Study 3)

## Overview

This is a budget-controlled hyperparameter tuning session
  for a Random Forest
classifier. The experiment count is tracked explicitly
  because this run will be
compared against classical hyperparameter optimisation
  methods (random search,
Bayesian optimisation) under the same budget. Every decision
  you make should
therefore be aimed at maximising the best val_loss achieved
  within the fewest
possible experiments      sample efficiency matters here as
  much as final quality.

## Setup

1. Agree on a run tag (e.g. 'rf-hp'). Create branch
  'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
  'train.py' in full.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
  user to run
  'uv run prepare.py'.
4. Create 'results.tsv' with header only. Add a column
  'exp_count' (integer,
  starting at 1) to track the cumulative experiment number.
5. Confirm readiness and await go-ahead.
```

Allowed hyperparameters

Change exactly one per experiment:

- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [3, 20] or None
- 'min_samples_leaf': integer in [1, 20]
- 'min_samples_split': integer in [2, 20]
- 'max_features': one of {"sqrt", "log2", 0.3, 0.5, 0.7}
- 'bootstrap': True or False
- 'class_weight': None or "balanced"

Do NOT change the model class, dataset, train/val split, 'prepare.py', or the evaluation function. No new packages.

The sole objective is to minimise val_loss.

Budget tracking

The total budget for this run is 60 experiments. After every 10 experiments, record a checkpoint summary comment in 'results.tsv' of the form:

```
"""--- checkpoint: best val_loss so far = X.XXXX at exp N ---"
```

Stop the loop and write a final summary when experiment 60 is complete, or continue if the user has not interrupted. The 60-experiment limit is a soft target for comparability; continue beyond it if the user allows.

Output format

```
'''
val_loss:          0.2314
training_seconds: 300.1
peak_vram_mb:      1820.4
'''
```

Extract: 'grep "^val_loss:" run.log'

Logging

Extended 'results.tsv' schema (tab-separated):

```
'''
exp_count  commit  val_loss  memory_gb  status  description
'''

Do not commit 'results.tsv'.

## Experiment loop

LOOP (target: 60 experiments):

1. Increment exp_count. Review 'train.py' and history.
2. Choose the single change most likely to improve val_loss
   given the history.
   Prioritise changes that provide the most information per
   experiment (i.e.
   prefer testing unexplored regions of the HP space over
   fine-tuning already-tested
   values).
3. Change exactly one hyperparameter.
4. 'git commit -m "[exp N] <HP: old->new, hypothesis>"'
5. 'uv run train.py > run.log 2>&1'
6. 'grep "^val_loss:" run.log'
7. On crash: log crash (exp_count increments), fix once if
   trivial, else move on.
8. Record in extended 'results.tsv'.
9. Keep if improved; 'git reset --hard HEAD~1' otherwise.

After experiment 60, write final summary. Then continue if
not interrupted.
NEVER STOP prematurely.
```

4.2 S3-GBM-HP Best GBM variant for HP tuning comparison

Variant ID: S3-GBM-HP **Algorithm:** Gradient Boosting (XGBoost) **Scope:** Narrow
Strategy: Sequential with interaction awareness **Dimension tested:** Budget-tracked;
 used in Study 3 comparison

Listing 14: S3-GBM-HP: file to save as program.md

```
# autoresearch      Gradient Boosting Hyperparameter Tuning
  (Study 3)

## Overview

This is a budget-controlled hyperparameter tuning session
  for a Gradient Boosting
classifier. The experiment count is tracked explicitly for
  comparison against
classical methods under the same budget. Sample efficiency
  is critical: each
experiment should be chosen to maximise the expected
  improvement in val_loss per
experiment consumed.

Gradient Boosting has stronger parameter interactions than
  Random Forest. The most
important interaction is learning_rate and n_estimators:
  lower learning_rate requires
more estimators. Test these individually first, then
  consider adjusting them jointly
once their individual effects are understood. Regularisation
  parameters (reg_alpha,
reg_lambda) should be explored after the core HP space is
  characterised.

## Setup

1. Agree on a run tag (e.g. 'gbm-hp'). Create branch
  'autoresearch/<tag>' from master.
2. Read 'README.md', 'prepare.py' (do not modify), and
  'train.py' in full.
3. Verify data in '~/.cache/autoresearch/'. If absent, ask
  user to run
  'uv run prepare.py'.
4. Create 'results.tsv' with extended header including
  'exp_count'.
5. Confirm and begin.

## Allowed hyperparameters
```

```

Change exactly one per experiment:
- 'n_estimators': integer in [50, 500]
- 'max_depth': integer in [2, 10]
- 'learning_rate': float in [0.005, 0.3]
- 'subsample': float in [0.5, 1.0]
- 'colsample_bytree': float in [0.5, 1.0]
- 'min_child_weight': integer in [1, 10]
- 'gamma': float in [0.0, 1.0]
- 'reg_alpha': float in [0.0, 1.0]
- 'reg_lambda': float in [0.5, 5.0]

Do NOT change the model class, dataset, 'prepare.py', or the
evaluation function.
No new packages. One change per experiment.

Sole objective: minimise val_loss.

## Budget tracking

Total budget: 60 experiments. Checkpoint every 10
experiments:
"--- checkpoint: best val_loss so far = X.XXXX at exp N ---"

Write a final summary at experiment 60, then continue if not
interrupted.

## Output format

'''
val_loss:          0.2201
training_seconds:  300.1
peak_vram_mb:      920.3
'''

Extract: 'grep "^val_loss:" run.log'

## Logging

Extended schema (tab-separated):

'''
exp_count  commit  val_loss  memory_gb  status  description
'''

Description format: "learning_rate 0.1->0.05 (interaction
note if relevant)"

```

```
Do not commit 'results.tsv'.

## Experiment loop

LOOP (target: 60 experiments):

1. Increment exp_count. Review history and identify the most
   informative change.
   Suggested order for a cold start: (1) scan learning_rate,
   (2) scan n_estimators,
   (3) scan max_depth, (4) scan subsample, (5)
   regularisation, (6) fine-tune best
   region. Deviate if history reveals a more promising
   direction.
2. Change exactly one hyperparameter.
3. 'git commit -m "[exp N] <HP: old->new, expected effect>"'
4. 'uv run train.py > run.log 2>&1'
5. 'grep "^val_loss:" run.log'
6. On crash: log crash, fix once if trivial, else move on.
   Count as experiment.
7. Record in extended 'results.tsv'.
8. Keep if val_loss improved; 'git reset --hard HEAD~1'
   otherwise.

After experiment 60 write final summary. Continue if not
interrupted.
NEVER STOP prematurely.
```


5. Quick Reference Summary

ID	Algorithm	Scope	Constraints	Strategy	Eval	Verbosity
S1-RF-1	Random Forest	Narrow	Strict	None	Single	Terse
S1-RF-2	Random Forest	Broad	Strict	None	Multi	Terse
S1-RF-3	Random Forest	Narrow	Permissive	None	Multi	Verbose
S1-RF-4	Random Forest	Broad	Permissive	None	Single	Verbose
S1-RF-5	Random Forest	Narrow	Strict	Sequential	Multi	Verbose
S1-RF-6	Random Forest	Broad	Strict	Sequential	Single	Verbose
S1-RF-7	Random Forest	Narrow	Permissive	Sequential	Single	Terse
S1-RF-8	Random Forest	Broad	Permissive	Sequential	Multi	Terse
S2-GBM-1	XGBoost	Narrow	Strict	None	Single	Terse
S2-GBM-2	XGBoost	Broad	Strict	None	Multi	Terse
S2-GBM-3	XGBoost	Narrow	Permissive	None	Multi	Verbose
S2-GBM-4	XGBoost	Broad	Permissive	None	Single	Verbose
S2-GBM-5	XGBoost	Narrow	Strict	Sequential	Multi	Verbose
S2-GBM-6	XGBoost	Broad	Strict	Sequential	Single	Verbose
S2-GBM-7	XGBoost	Narrow	Permissive	Sequential	Single	Terse
S2-GBM-8	XGBoost	Broad	Permissive	Sequential	Multi	Terse
S3-RF-HP	Random Forest	Narrow	Strict	Sequential	Single	Verbose
S3-GBM-HP	XGBoost	Narrow	Strict	Sequential	Single	Verbose

Notes on Study 2 variants S2-GBM-3, S2-GBM-4, S2-GBM-6, S2-GBM-7. These four variants are structurally identical to their Study 1 RF counterparts (S1-RF-3, S1-RF-4, S1-RF-6, S1-RF-7) with two substitutions: (a) the model class is `XGBClassifier` instead of `RandomForestClassifier`, and (b) the allowed hyperparameter list is the GBM list from S2-GBM-1. The remainder of each document — setup procedure, output format, logging schema, loop logic — is unchanged. To produce these four files, take the corresponding Study 1 listing and apply the two substitutions above.

Notes on Study 3 variants. The HP space in S3-RF-HP and S3-GBM-HP is deliberately narrow to make the comparison with classical Bayesian optimisation fair: Optuna and random search are also restricted to the same enumerated HP space. The budget of 60 experiments corresponds to the wall-clock time of a short overnight run and is the matching budget used for all classical baselines.